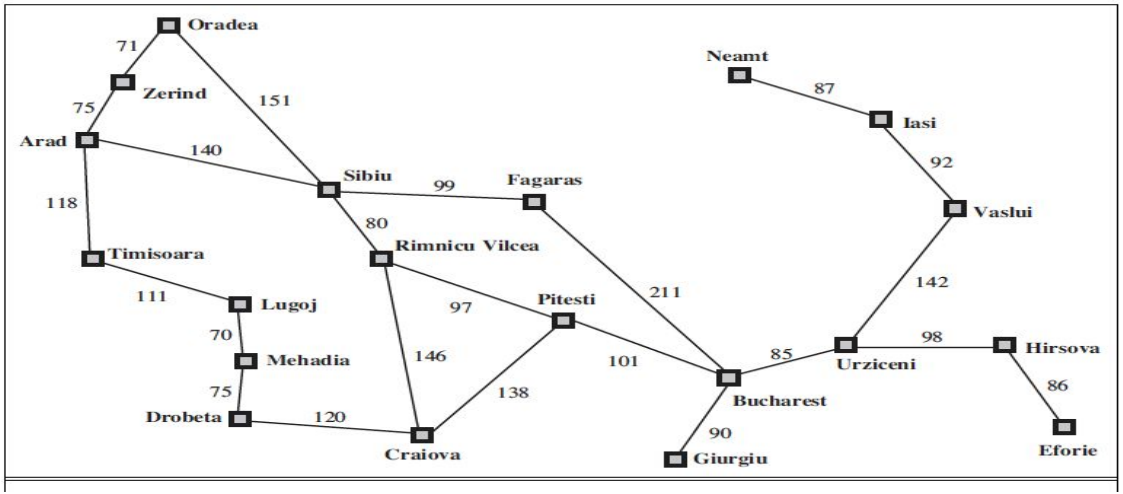
In Greedy Best First Search

- We are ignoring the cost to the current path and only looking to the extra cost to give to the goal
- In GBFS, we don't consider g(n)
- f(n) = h(n)
- Is this ideal?



A simplified road map of part of Romania

- Our objective is to go from the start state to the goal, not from the goal.
- The state ***n*** is somewhere in the middle, but our objective is to go from the start state to the goal and
- as we are taking the minimum, we should not myopically only take the minimum concerning how much cost we are going to pay in the future, but also take minimum over the cost that we have paid so far.

A* Search

A* is a widely used search algorithm in computer science, particularly for pathfinding and graph traversal problems. It is an informed search algorithm that balances the need to find the shortest path (optimality) with the need to find it efficiently (speed).

Idea: **avoid expanding paths that are already expensive**

The Combined Cost Function

At the heart of the A* algorithm is the combined cost function ***f(n)***, which determines which node to expand next:

$$f(n) = g(n) + h(n)$$

Where:

- **g(n):** The actual cost from the start node to the current node n.
- **h(n):** The heuristic estimate of the cost from the current node n to the goal node. This is an estimate based on domain-specific knowledge.
- **f(n):** The total estimated cost of the cheapest solution that passes through the current node nnn, combining both the known cost to reach nnn and the estimated cost from nnn to the goal.

The A* algorithm efficiently balances the exploration of nodes by considering both the actual cost incurred so far and an estimate of the remaining cost to reach the goal:

Role of g(n) (Actual Cost):

- **Accurate Path Cost:**
  - The g(n) term ensures that the algorithm accounts for the real cost of reaching a particular node from the start node. This means that paths that have lower cumulative costs are favored, helping to ensure that the algorithm does not choose paths that are deceptively cheap in the short term but expensive overall.
- **Accumulation of Costs:**
  - g(n) is calculated by summing the costs of all edges along the path from the start node to the current node n. As the algorithm progresses, g(n) accumulates, representing the true cost of the path taken so far.

Role of h(n) (Heuristic Estimate):

- **Guidance Toward the Goal:**
  - The heuristic h(n) provides an estimate of the remaining cost to reach the goal. It guides the search process by directing it toward nodes that appear closer to the goal, thus reducing the number of nodes that need to be explored.
- **Influence on Efficiency:**
  - The quality of the heuristic h(n) greatly affects the efficiency of the A* algorithm. A good heuristic (one that closely approximates the true remaining cost) will speed up the search by focusing on the most promising paths. However, if the heuristic is poor, the algorithm may end up exploring unnecessary nodes, reducing efficiency.

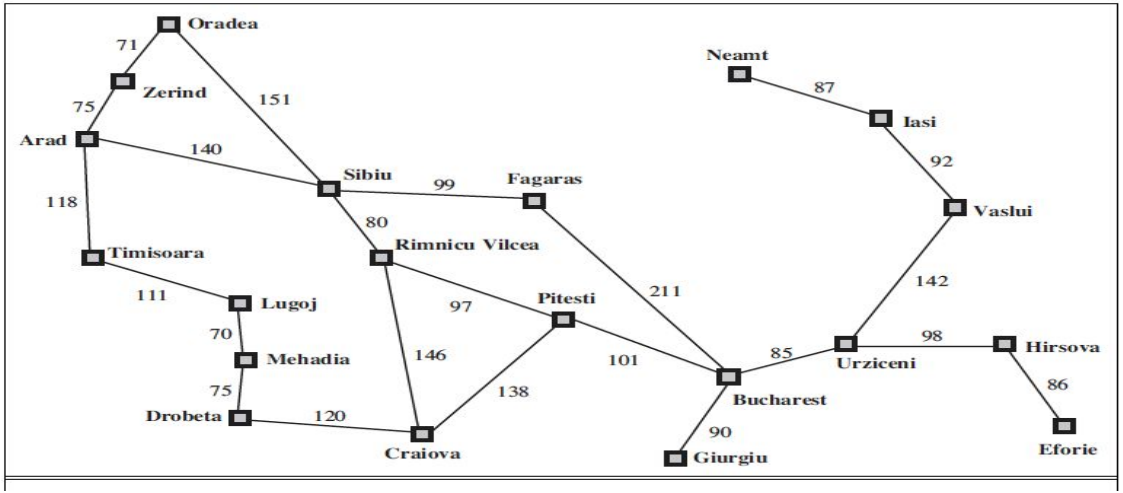
The Balance:

- **Optimality vs. Speed:**
  - A* achieves a balance between g(n) and h(n) by combining them into the f(n) function. This combination allows A* to find the shortest path efficiently by considering both the actual cost of reaching a node and an estimate of the cost to complete the path.
  - The algorithm prioritizes nodes with lower f(n) values, effectively balancing exploration of new paths (guided by h(n)) and exploitation of known paths (tracked by g(n)).
- **Admissibility and Consistency:**
  - For A* to be both optimal and complete, the heuristic h(n) must be admissible (never overestimating the true cost to reach the goal) and consistent (ensuring that the estimated cost from any node to the goal is less than or equal to the cost of reaching a neighbor plus the estimated cost from the neighbor to the goal).

It is not only looking at h, which is greedy best-first search. It is not only looking at g, which is a uniform cost search

A* for Romanian Shortest Path

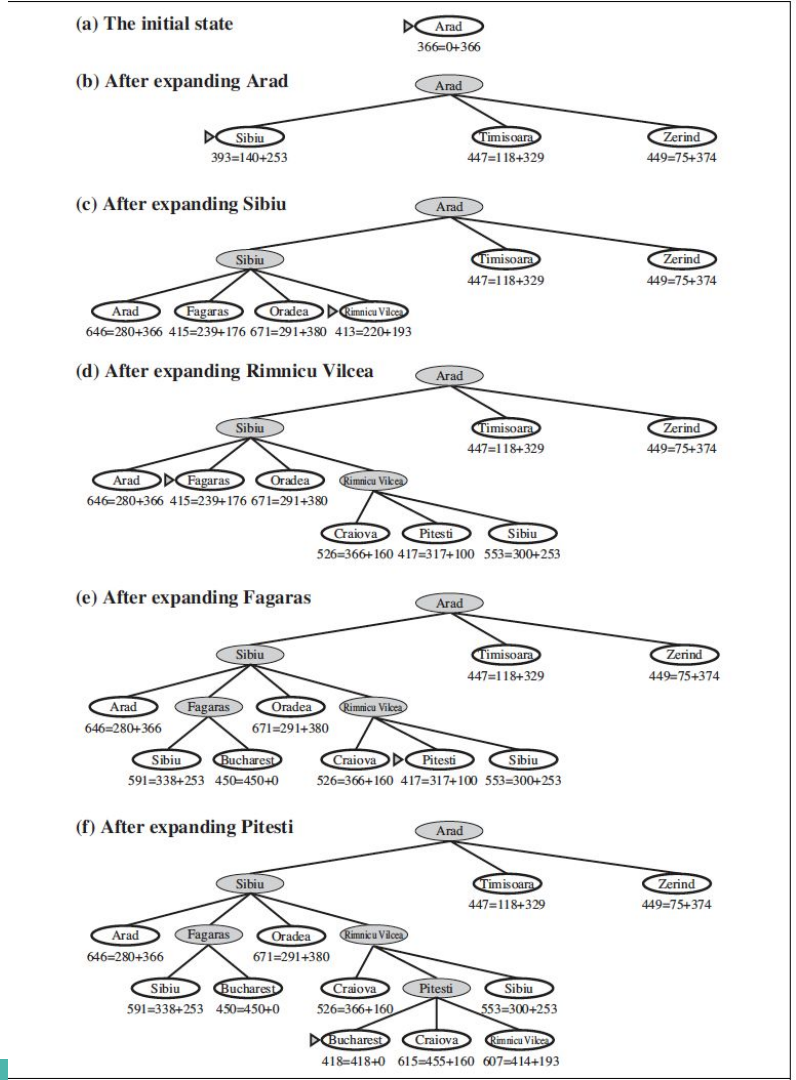
Romanian Map



Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Values of h_{SLD}—straight-line distances to Bucharest

Stages in an A* search for Bucharest. Nodes are labeled with f = g +h. The h values are the straight-line distances to Bucharest



Algorithm Steps

Step 1: Initialize the Open and Closed Lists

- **Open List:** This is a priority queue that contains the nodes to be explored. The nodes in the open list are sorted based on their f(n)=g(n)+h(n) value, where g(n) is the actual cost from the start node to node n, and h(n) is the heuristic estimate of the cost from n to the goal.
- **Closed List:** This is a set that contains the nodes that have already been explored and should not be revisited.

Step 2: Add the Start Node to the Open List

- Initialize g(start)=0 because the cost to reach the start node from itself is zero.
- Calculate h(start), the heuristic estimate of the cost from the start node to the goal.
- Set f(start) = g(start) + h(start).
- Add the start node to the open list.

Step 3: Repeat Until the Goal is Reached or the Open List is Empty

While the open list is not empty, perform the following steps:

Step 3.1: Select the Node with the Lowest f(n) Value

- Remove the node n from the open list that has the lowest f(n) value.
- If n is the goal node, the algorithm terminates, and the path is reconstructed from the goal to the start node by following parent pointers.

Step 3.2: Generate Successors

- For each neighbor (successor) of the current node n:
  - Calculate g(neighbor) = g(n) + cost(n,neighbor), where cost(n,neighbor) is the actual cost to move from n to the neighbor.
  - Calculate h(neighbor), the heuristic estimate of the cost from the neighbor to the goal.
  - Set f(neighbor)=g(neighbor)+h(neighbor).

Step 3.3: Check if the Neighbor Should be Added to the Open List

- If the neighbor is already in the closed list, skip it.
- If the neighbor is not in the open list, add it and set the current node n as its parent.
- If the neighbor is in the open list but the new path to it has a lower g(neighbor) value, update its g(neighbor) and f(neighbor) values and change its parent to the current node n.

Step 3.4: Add the Current Node to the Closed List

- After all neighbors have been evaluated, add the current node n to the closed list to ensure it is not revisited.

Step 4: Reconstruct the Path

- If the goal node was reached, reconstruct the path by following the parent pointers from the goal node back to the start node.

Goodness of Heuristic Functions

- Admissibility
- Consistency

Admissible Heuristics: Definition of Admissibility

A heuristic function $h(n)$ is said to be **admissible** if it never overestimates the true cost of reaching the goal from any node n in the search space.

Mathematically, this is expressed as:

$$h(n) \leq h^*(n) \quad \text{for all nodes } n$$

Where:

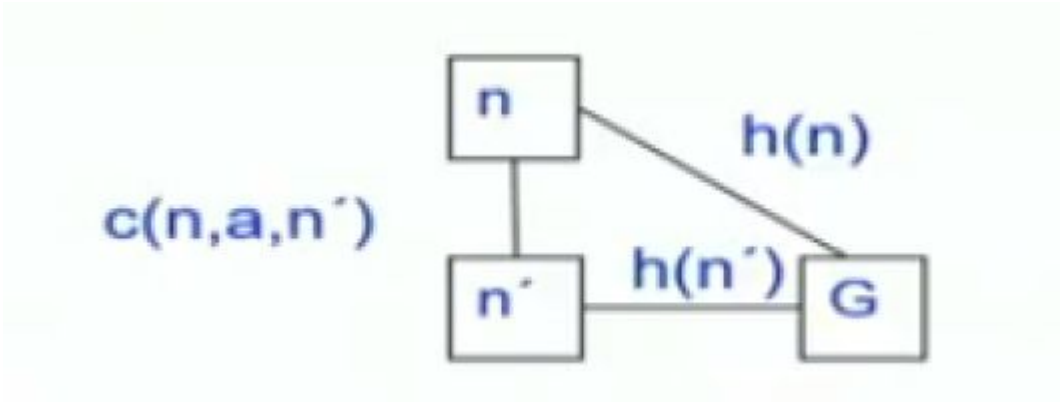
- $h(n)$ is the heuristic estimate of the cost to reach the goal from node n.
- $h^*(n)$ is the actual cost of the optimal path from node n to the goal.

- An admissible heuristic is **Optimistic**
- Example: h_{SLD} (never overestimates the actual road distance)
- What is most optimistic?
  - What will be the heights of optimism for the Node?
  - The Node will not need to search for the Goal (I am goal) as it is goal itself, right?
  - In other words, h will be equal to 0 if I am the goal, then the distance to myself is 0.
  - So the most optimistic heuristic would be 0.
  - However, what would be the meaning of optimism? The meaning of optimism would be that the cost to the goal is estimated as less than what it is. You feel that I will have to pay less cost. Then you end up paying. That is the meaning of admissibility.
- If, however, you are in a place where you want to make money and higher is better, what is an admissible heuristic one that makes you believe that you will earn more, not less.
- Admissible heuristic is not always that “less is optimal”. It is less than or less than equal to optimal for minimization problems, but it is greater than equal to optimal for a maximization problem.

- **Theorem**: If h(n) is admissible, then the Tree-Search version of A* is optimal.
- The interesting thing is that if it's the graph search A* version, then admissibility is insufficient.
- It is a necessary condition but not a sufficient condition.
- When would the graph search version of A* be optimal?
- New condition is needed called **consistency**

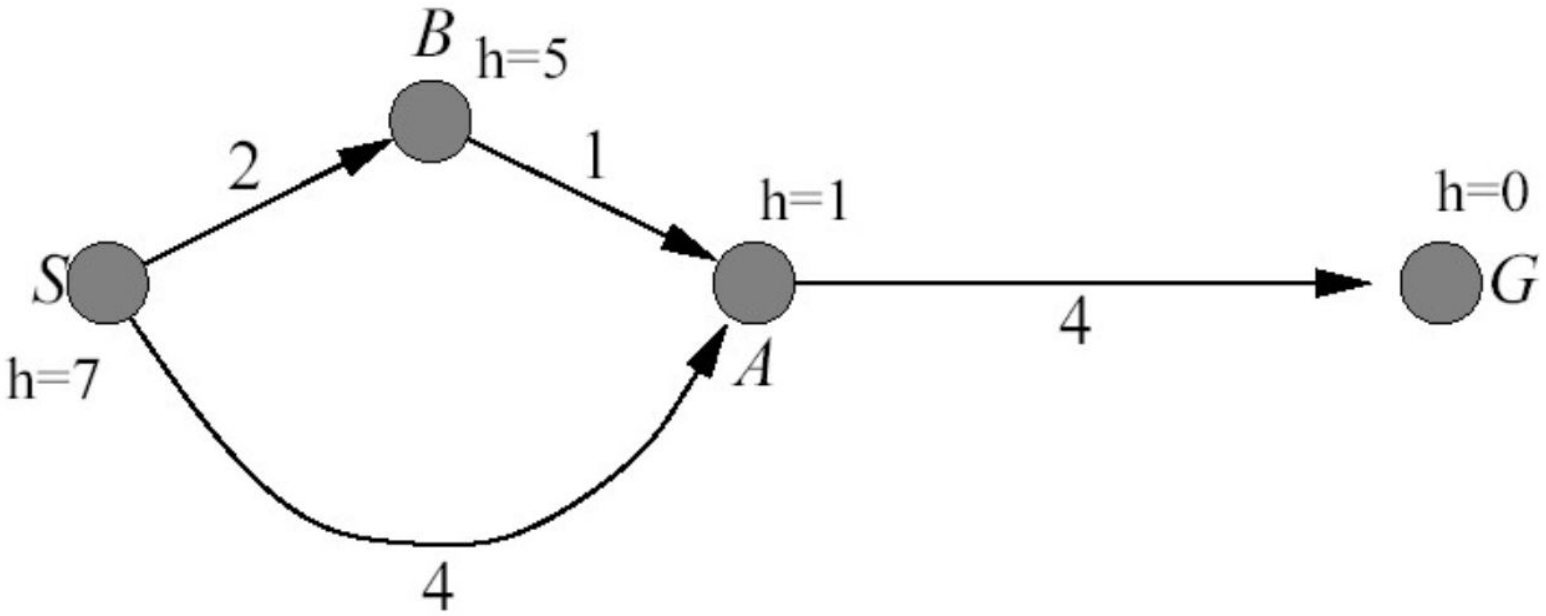
Consistent Heuristics (Triangle Inequality)

- h(n) is consistent if
  - for every node **n**
  - for every successor **n'** due to legal action **a**
  - **h(n) <= c(n,a,n') + h(n')** (triangle inequality)



- Every consistent heuristic is also admissible, i.e, consistency subsumes admissibility
- Theorem: If h(n) is consistent, A* using Graph-Search is optimal.

Example: Graph-Search version of A* having Admissible Heuristics



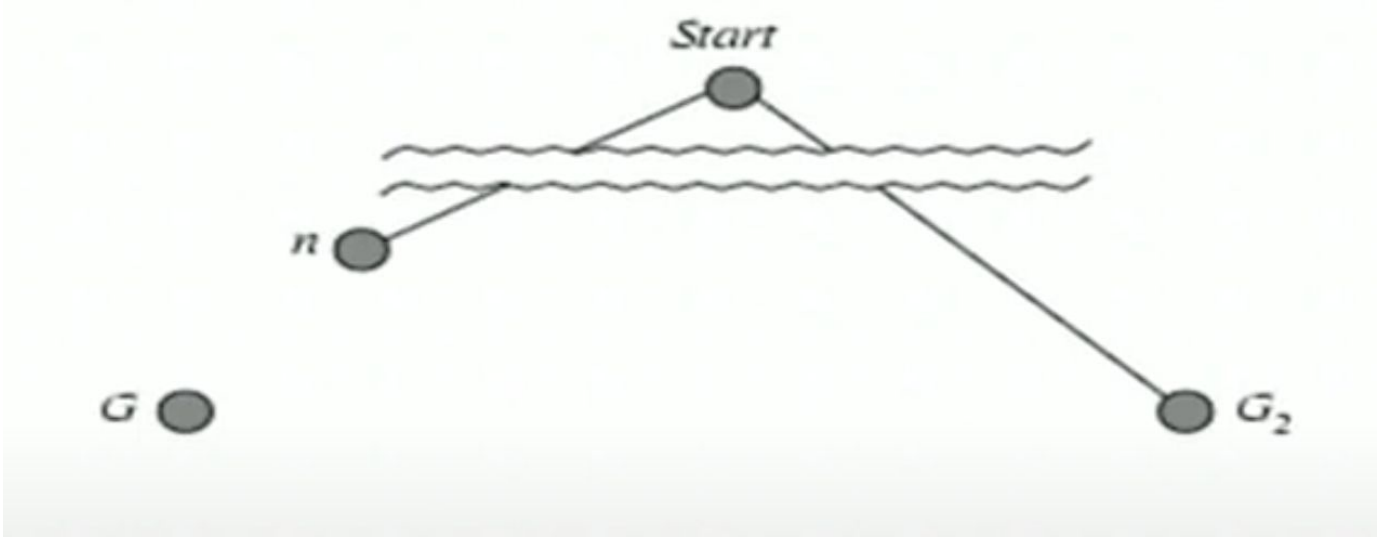
https://stackoverflow.com/questions/25823391/suboptimal-solution-given-by-a-search

Proof of Optimality of A* (Tree Version)

- We are trying to look at how to prove optimality for the A^* algorithm when the heuristic is admissible.
- Assume ***h()*** is admissible
- Say some sub-optimal goal state G_2 has been generated and is on the frontier and we are about to remove it.
- Currently, we have a start node ***S*** we are about to remove G_2 from the fringe.
- We believe that the algorithm should finish, and we have found the optimal path.

- Let us say that it is not the optimal path, i.e., there is another path which is the optimal path.
- If it is another path, it would have it's goal node, so let us say the optimal goal now that we are looking for is G.
- And either G may be in the fringe or some other node to that path has to be in the fringe at this point.
- Let ***n*** be an unexpanded state such that ***n*** is on the optimal path to the optimal goal ***G***.

- Let us say that path is this one in the diagram, and let us say that we have to have some node in the fringe that leads us to this goal ***G***.
- We chose to remove G_2 and not remove n.
- So if the algorithm was right, it should have removed ***n***, but the algorithm tried to remove G_2
- That means, for some reason, the algorithm thought that G_2 was the right node to remove, which means its F value of G_2 was less than the F value of ***n***.



Let's focus on G_2,

- $f(G_2) = g(G_2)$ [since $h(G_2) = 0$]
- $g(G_2) > g(G)$ [since G_2 is suboptimal]

Now focus on G,

- $f(G) = g(G)$ [since $h(G) = 0$]
- $f(G_2) > f(G)$ [substitution in above 3 equations]

Now let's focus on ***n***,

- $h(n) \leq h^*(n)$ [since h is admissible]
- $g(n) + h(n) \leq g(n) + h^*(n)$ [algebra]
- $f(n) = g(n) + h(n)$ [by definition]
- $f(G) = g(n) + h^*(n)$ [by assumption]
- $f(n) \leq f(G)$ [substitution using $f(G_2) > f(G)$]

Hence $f(G_2) > f(n)$, and A* will never select G_2 for expansion (therefore it is a contradiction)

Proof of Completeness

Theorem: A* is complete, meaning it will find a solution if one exists, provided the search space is finite and the heuristic h(n) is admissible.

Proof:

- **Assumption:** The search space is finite, and the heuristic h(n) is admissible, i.e., $h(n) \leq h^*(n)$ for all nodes n, where $h^*(n)$ is the true cost from n to the goal.
- **Proof by Contradiction:**
  - Suppose A* is not complete, i.e., it fails to find a solution even though one exists.
  - This would imply that A* exhausts all nodes in the open list without finding the goal, which means the open list eventually becomes empty.
  - For A* to fail, it must be the case that all possible paths to the goal have been pruned or ignored.
  - However, because the heuristic h(n) is admissible, A* will not ignore any potential path that could lead to the goal. The admissible heuristic ensures that A* explores all feasible paths until the goal is found, as the estimated cost $f(n) = g(n) + h(n)$ will always lead A* to explore paths that can reach the goal.
- **Conclusion:** Since A* will eventually explore every possible path that could lead to the goal, it must find the goal if one exists, proving that A* is complete.

Properties of A* Search

1. **Optimality**:
   - A* guarantees finding the least-cost path if the heuristic h(n) is admissible and consistent. This means it never overestimates the actual cost to reach the goal.
2. **Completeness**:
   - A* is complete; it will find a solution if one exists, provided there is enough memory to keep track of all nodes.
3. **Efficiency(Time)**:
   - A* can be very efficient if a good heuristic is used, leading to fewer nodes being expanded compared to uninformed search strategies. In worst case, all nodes are expanded.
4. **Memory Usage(Space)**:
   - A* stores all generated nodes in memory, which can lead to high memory consumption, especially in large search spaces.

Advantages of A* Search

1. **Optimality and Completeness**: A* guarantees finding the optimal solution, making it a reliable choice for many applications.
2. **Flexibility**: A* can be adapted with different heuristic functions to fit various problem domains.
3. **Wide Applicability**: It is widely used in pathfinding, game development, robotics, and various optimization problems.

Disadvantages of A* Search

1. **Memory Intensive**: A* can require significant memory to store all generated nodes, particularly in large or complex search spaces.
2. **Performance Depends on Heuristic**: If the heuristic is poor or poorly tuned, A* can behave like a breadth-first search, resulting in inefficient performance.
3. **Computational Overhead**: The overhead of calculating and maintaining g(n) and h(n) values can add to the computational cost.

Bidirectional A* Search

Bidirectional A* Search is an extension of the A* search algorithm that seeks to improve search efficiency by simultaneously exploring paths from both the start node and the goal node. This approach aims to reduce the search space and the number of nodes that need to be expanded, making it especially useful in large search spaces.

Key Features of Bidirectional A* Search

1. **Two Search Frontiers**: The algorithm maintains two open lists: one for the forward search (from the start node) and one for the backward search (from the goal node).
2. **Meeting Point**: The searches continue until the two frontiers meet, at which point the algorithm can construct a solution path.
3. **Heuristic Function**: Like A*, Bidirectional A* uses heuristic functions to guide both the forward and backward searches, ensuring that both searches are informed.

How Bidirectional A* Search Works

1. **Initialization**:
   - Start with two open lists: one for the forward search from the start node and one for the backward search from the goal node.
   - Initialize the cost values g and heuristic values h for both searches.
2. **Node Expansion**:
   - Perform the following steps until one of the open lists is empty:
     - Expand the node with the lowest f(n)=g(n)+h(n) from the forward search.
     - If this node is found in the backward search's open list, a path has been discovered.
     - If not, generate its successors and add them to the forward open list.
3. **Simultaneous Search**:
   - In the same iteration, expand the node with the lowest f(n) from the backward search.
   - If this node is found in the forward search's open list, a path has been discovered.
4. **Meeting Point**:
   - When a node from either search connects to a node from the other search (i.e., the same state is found), a solution path can be constructed.
   - The total cost of the solution can be computed by combining the costs from both searches.
5. **Path Construction**:
   - Trace back from the meeting node to construct the complete path from the start node to the goal node by combining the paths from both searches.

Properties of Bidirectional A* Search

1. **Efficiency**:
   - By searching from both the start and goal simultaneously, Bidirectional A* can significantly reduce the search space, especially in large graphs or grids. It effectively narrows down the number of nodes expanded.
2. **Optimality**:
   - Bidirectional A* is guaranteed to find the optimal solution as long as both searches use admissible heuristics.
3. **Completeness**:
   - The algorithm is complete; it will find a solution if one exists, provided there is sufficient memory.
4. **Memory Usage**:
   - Bidirectional A* requires maintaining two open lists, which can increase memory usage compared to a unidirectional A* search. However, it may still use less memory overall since it expands fewer nodes.

Advantages of Bidirectional A* Search

1. **Reduced Search Time**: The simultaneous search can lead to faster solutions, particularly in large state spaces.
2. **Improved Time Complexity:** The reduced search space often translates into a reduction in time complexity. By expanding fewer nodes, Bidirectional A* can reach the goal faster than unidirectional A*.
3. **Potential for Faster Convergence:** Since each search frontier only needs to reach approximately halfway to the goal, the chances of the two frontiers meeting quickly are higher. This can lead to faster discovery of the optimal path.
4. **Balanced Exploration:** By balancing exploration between the forward and backward searches, Bidirectional A* can prevent the algorithm from getting "stuck" in certain parts of the search space. If one search direction becomes less promising, the other can compensate.

Limitations of Bidirectional A*

Heuristic Consistency and Symmetry:

- The success of Bidirectional A* heavily depends on the heuristics being consistent and symmetric (i.e., $h_f(n)=h_b(n)$). If the heuristics are not well-matched, one of the searches may dominate, reducing the benefit of bidirectional search.

Meeting Point Detection:

- Detecting the exact meeting point where the two searches should merge can be complex, especially in cases where the search frontiers do not meet at a single node but rather in a region of nodes.

Space Complexity:

- Although bidirectional search reduces time complexity, the space complexity remains high. The algorithm still requires storing all expanded nodes in both directions, which can be memory-intensive.

Implementation Complexity:

- Implementing bidirectional search is more complex than unidirectional search. Managing two separate frontiers, ensuring heuristic consistency, and correctly merging the paths add to the implementation difficulty.

Path Quality in Weighted Graphs:

- In weighted graphs, ensuring that the path found by bidirectional A* is truly optimal can be challenging if the forward and backward costs are not carefully managed. The weights might distort the balance between the two searches.

Iterative Deepening A* Search

Iterative Deepening A* (IDA*) is an informed search algorithm that combines the benefits of A* search and iterative deepening search. It is particularly useful for problems where the search space is large and the depth of the solution is not known in advance. IDA* systematically explores paths to a certain depth, using a heuristic to guide the search, while iteratively increasing the depth limit until a solution is found.

Key Features of Iterative Deepening A* Search

1. **Combination of Depth-First and Best-First Search**: IDA* uses a depth-first search approach to explore the search space while incorporating a heuristic to evaluate the cost to reach the goal.
2. **Memory Efficiency**: IDA* uses significantly less memory compared to A*, as it does not store all generated nodes in memory. Instead, it only keeps track of the current path.
3. **Optimality**: IDA* is guaranteed to find the optimal solution if the heuristic used is admissible.
4. **Completeness**: IDA* is complete; it will find a solution if one exists, provided that the search space is finite.

How Iterative Deepening A* Search Works

IDA* follows a systematic approach to explore the search space. The main components include:

1. **Evaluation Function**:
   - Similar to A*, IDA* uses an evaluation function f(n) defined as:

     $$f(n)=g(n)+h(n)$$

   - Where:
     - g(n): The actual cost from the start node to node n.
     - h(n): The heuristic estimate of the cost from node n to the goal.
2. **Depth-Limited Search**:
   - The algorithm conducts a depth-first search with a depth limit defined by the current threshold. This threshold is incrementally increased after each complete iteration.
3. **Threshold Management**:
   - The algorithm starts with a threshold based on the heuristic value of the start node, f(start)=h(start).
   - If a node exceeds this threshold during the search, the algorithm returns and increments the threshold, repeating the process until a solution is found.
4. **Recursive Exploration**:
   - During each iteration, the algorithm recursively explores nodes up to the current depth limit. If a node is reached that exceeds the threshold, it backtracks and tries other nodes.
   - Nodes that lead to a solution are recorded, allowing the algorithm to construct the solution path once the goal is found.

Steps of Iterative Deepening A* Search

1. **Initialization**:
   - Start with the initial node (the starting state).
   - Set the initial threshold to f(start) (this can be zero or based on h(start)).
   - Initialize the solution variable to keep track of the best cost found.
2. **Depth-Limited Search**:
   - Perform a depth-first search with the current threshold:
     - Expand the node with the lowest f(n) value.
     - If the goal node is reached, update the solution and store the cost.
     - If a node's f(n) exceeds the threshold, backtrack and return the threshold value.
3. **Update Threshold**:
   - After completing the depth-limited search for the current threshold:
     - If a solution is found, the algorithm terminates.
     - If not, increment the threshold to the lowest f(n) value of the nodes that exceeded the threshold during the search and repeat the depth-limited search.
4. **Repeat**:
   - Continue this process, incrementing the threshold until a solution is found.

Properties of Iterative Deepening A* Search

1. **Optimality**:
   - IDA* guarantees finding the optimal solution if the heuristic is admissible and consistent.
2. **Completeness**:
   - The algorithm is complete, meaning it will find a solution if one exists within a finite search space.
3. **Memory Efficiency**:
   - Unlike A*, IDA* does not need to store all nodes, making it suitable for large search spaces.
4. **Performance**:
   - IDA* may perform better than A* in some scenarios, especially when the solution depth is shallow compared to the size of the search space.

Advantages of Iterative Deepening A* Search

1. **Memory Usage**: IDA* uses much less memory than A*, making it feasible for large problems.
2. **Guaranteed Optimality**: The algorithm guarantees finding the optimal solution when using an admissible heuristic.
3. **Simplicity**: IDA* can be easier to implement than maintaining a priority queue in A*.

Limitations of IDA*

Repeated Work:

- A major drawback of IDA* is that it performs repeated work across iterations. Nodes near the root of the search tree are expanded multiple times, which can significantly increase the time complexity compared to A*.

Performance on Complex Heuristics:

- For problems where the heuristic function is complex or expensive to compute, the repeated evaluations in each iteration can make IDA* inefficient.

Lack of Path Reconstruction:

- Unlike A*, which can reconstruct the path as it searches, IDA* only finds the path after reaching the goal. This can make it more difficult to provide intermediate solutions or progress updates.

Threshold Management:

- Managing the threshold updates can be tricky, particularly in problems with varying step costs. If the increments are too small, many unnecessary iterations may occur; if too large, the search may miss the optimal solution.

Memory-Bounded A* Search

Memory-Bounded A* search algorithms are designed to address the high memory requirements of the standard A* algorithm by limiting the amount of memory used during the search process. The basic idea is to retain the benefits of A* (such as finding optimal paths) while ensuring that the algorithm can run even when memory is limited.

Key Concepts

1. **Memory Limitation:**
   - Unlike A*, which keeps all generated nodes in memory, Memory-Bounded A* algorithms impose a limit on the number of nodes that can be stored at any given time. This is crucial in environments where memory is a critical constraint.
2. **Node Pruning:**
   - When the memory limit is reached, Memory-Bounded A* algorithms must prune or discard nodes to make room for more promising ones. The choice of which nodes to prune is key to maintaining optimality and efficiency.
3. **Re-expansion of Nodes:**
   - If a pruned node turns out to be necessary for finding the optimal path, Memory-Bounded A* algorithms may need to re-expand it. This re-expansion can lead to increased computational effort, but it's necessary to ensure that the best path is found.

Variants of Memory-Bounded A*

1. MA* (Memory-Bounded A*)

MA* is an early form of Memory-Bounded A* that introduces a simple mechanism for handling memory constraints:

- **Memory Limit:** MA* keeps track of the nodes in memory and discards the least promising ones when the memory limit is reached. The nodes are typically evaluated based on their ***f(n)=g(n)+h(n)*** value, where ***g(n)*** is the cost to reach the node and ***h(n)*** is the heuristic estimate to reach the goal.
- **Node Pruning:** When pruning is necessary, MA* removes the nodes with the highest ***f(n)*** values (i.e., the least promising nodes). These nodes might be needed later, so their best ***f*** values are stored as backup.
- **Re-expansion:** If the search later determines that a pruned node should be revisited (e.g., because it led to a potentially better solution), MA* must re-expand that node. This can result in increased time complexity due to the repeated expansion of nodes.
- **Optimality:** MA* can find the optimal solution if given enough time, but the need for re-expansion can lead to inefficiencies.

2. SMA* (Simplified Memory-Bounded A*)

SMA* improves upon MA* by offering a more sophisticated approach to managing memory:

- **Memory Management:** SMA* maintains a fixed amount of memory and uses it to store the most promising nodes, based on their ***f(n)*** values. Unlike MA*, SMA* tries to minimize the need for re-expansion by keeping track of backup paths.
- **Pruning Strategy:** When the memory limit is reached, SMA* removes the least promising nodes, similar to MA*. However, SMA* also keeps track of the best alternative path for each pruned node, which reduces the need for re-expansion.
- **Backtracking:** SMA* can backtrack when it encounters a dead end or when all remaining paths have ***f(n)*** values higher than those of previously pruned nodes. This backtracking allows SMA* to explore alternative paths without significantly increasing memory usage.
- **Optimality:** SMA* is optimal, meaning it will find the best solution within the given memory constraints. It is designed to handle the memory limit more efficiently than MA*, leading to better performance in practice.

How Memory-Bounded A* Works

1. **Initialization:**
   - The algorithm starts by initializing the open list (frontier) with the start node, just like A*.
   - The memory limit is defined, which restricts the number of nodes that can be stored in the open list at any time.
2. **Node Expansion:**
   - Nodes are expanded based on their f(n)=g(n)+h(n) value. The node with the lowest f(n) is expanded first, following the same principle as A*.
3. **Memory Check:**
   - After expanding a node, the algorithm checks whether the number of nodes in memory exceeds the predefined limit.
   - If the memory limit is not reached, the algorithm continues expanding nodes.
4. **Pruning:**
   - When the memory limit is reached, the algorithm must prune nodes to make room for more promising ones.
   - The least promising node (the one with the highest f(n) value) is pruned first.
   - The pruned node's f value is stored in case the node needs to be re-expanded later.
5. **Backtracking (SMA):***
   - If all paths from the current node lead to dead ends or paths with higher f(n) values than those of pruned nodes, SMA* will backtrack to explore alternative paths.
   - SMA* can re-expand pruned nodes if necessary, but it tries to minimize this by keeping track of backup paths.
6. **Goal Test:**
   - The algorithm continues expanding and pruning nodes until it finds the goal node.
   - When the goal is found, the path from the start node to the goal is reconstructed.

Limitations of Memory-Bounded A*

Increased Time Complexity:

- The need to prune and potentially re-expand nodes can lead to increased computational effort, making Memory-Bounded A* slower in some cases compared to standard A*.

Implementation Complexity:

- Managing memory and ensuring optimality within constrained memory limits can be complex, requiring careful implementation.

Suboptimality in MA:*

- MA* may not always find the optimal path if re-expansion is not handled correctly, leading to potential suboptimal solutions.